

Milestone 工程说明（工程地图・模式与 UI 更改・引擎改动・打包与命令）

适用对象：维护者/进阶用户・手册版本：v1.0（对应 t62 四目录迁移后布局）

手册源：编辑指南\文档精选（folder-structure-design / engine-unity-arch / engine-tool-design / engine-capability-list / 还原度闭环总结 / editor-trilab / layout-custom-design 等），文件路径均为**四目录新树**。

姊妹篇：《零基础更改指南》（编辑指南\零基础更改指南.md）——面向第一次改代码的用户，先读那篇完成 3 个动手任务后再回到本文档。

目录

- 1. 工程地图：四目录树与关键文件职责
- 2. 模式更改：加/改一个音游模式
- 3. UI 更改：改文字 / 布局 / 主题 / 图标
- 4. 引擎改动：Ruleset / 判定 / 渲染 / 引擎工具 CLI
- 5. 打包与命令：build / publish / pack / 快捷方式
- 6. 文档索引与路径映射
- 7. 附录：运行/调试命令速查与常见坑

1. 工程地图：四目录树与关键文件职责

1.1 四目录树（迁移后）

~~~

- D:\Users\etgya\Desktop\milestone\
- ├─ 游戏源码\ 会编译的一切（游戏层工程）
    - | └─ Milestone.csproj 主工程（WinExe/net8.0-windows；ProjectReference → 引擎源码\engine\MilestoneEngine.csproj；
    - | | OutputPath=..\输出产物\bin\；Samples Include=..\引擎源码\engine\Samples\）
    - | └─ Program.cs CLI 入口（--selfcheck/--parity/--shellshot/--pack 等全部命令行）
    - | └─ app.manifest 应用清单（DPI/权限）
    - | └─ 源码\ 游戏层源码树（见 1.2）

| └─ obj\ 本工程中间物

└─ 引擎源码\ 引擎类库与引擎工具工程

| └─ engine\ MilestoneEngine.csproj (类库) + 域根 \*.cs (RhythmCore/JudgementTracker/...) + App\ (宿主/ViewportPolicy)

| + Ui\ (UiComponents/UiTheme/UiCanvas) + Platform\ (SoftwareRenderer/Win32Window/GDI+D2D 适配)

| + Tests\EngineChecks\ (自检 CLI) + EnginePlay\ (引擎无头播放/预设闭环 CLI)

| + PublishPlayer\ (引擎单文件 exe) + HelloEngine\ (最小示范) + Samples\ (演示/示例, 编入游戏)

└─ 编辑指南\ 本手册 + 文档精选 + 工具脚本 + PDF

| └─ README.md 门户 (快速开始 + 路径映射 + 文档索引)

| └─ 工程说明.md 本文档

| └─ 零基础更改指南.md 面向零基础 (5 个动手任务)

| └─ 文档精选\ 历史协作报告/设计文档全量 (90+ 篇, 正文保留旧路径, 见 6.2 映射)

| └─ 工具脚本\\_min\*.ps1 (开发小脚本) 与打包命令.txt

| └─ PDF\ 工程说明.pdf / 零基础更改指南.pdf (md2html + Edge print-to-pdf 生成)

└─ 输出产物\ 一切产物

└─ bin\ 开发构建 (csproj OutputPath 指向: Release\net8.0-windows\win-x64\Milestone.exe)

└─ 发布\ 正式发布件 (Milestone.exe 单文件 + skin.json; Chart\Player\PlayerData\Replay\Log\ 运行时内容根)

└─ 单文件\ 额外打包产物 (MilestonePlayer.exe 等)

└─ 示例内容\ 素材库: Milestone示例\*.mil (九模式示例谱)、实例谱面、段位文件、测试格式、皮肤\osu\_default\

└─ dev\ 证据/压测输出 (历史取证目录与 caprepro/geo-test 小工具)

└─ 日志\ 根级日志归档

~~~

要点:

- **游戏内容根 = 程序当前目录 (BaseDirectory)**: 发布件 exe 旁自动生成 Chart\ (谱面, AppConfig.DefaultChartsFolder)、PlayerData\、Player\ (皮肤/玩家/训练)、Replay\、Log\、屏幕采集、人类试玩。把谱面丢进 Chart\ 即被曲库扫描到 (默认目录); 也可在曲库管理页「选择文件夹」指向任意目录。
- **运行时目录全部跟随 exe** (AppDomain.BaseDirectory 派生), 移动 exe 不丢配置——这是四目录迁移的硬约定 (见 文档精选\folder-structure-design.md §2-C) 。

1.2 游戏层关键文件表

文件 (游戏源码\源码\... 相对)	职责	改它做什么
Program.cs	CLI 入口: 全部命令行参数 (见 5.2)	加/改命令行功能
CoreUtil\UiText.cs	全部用户可见文案的唯一集中地 (菜单/设置/编辑器/次级页/结算...常量表)	改任何界面文字 (禁止在别处硬编码字符串)
CoreUtil\ModeSystem.cs	模式注册表 ModelInfo (Mode/Id/Display/Keys/格式/ADJUSTABLE...)	注册新玩法模式 (见 2.1)
CoreUtil\AppConfig.cs	配置模型 (ChartsFolder/Player/RecentCharts...) + config.json 读写	改默认目录/配置项 (含 t62 迁移标记)
CoreUtil\Logger.cs	会话日志 (Log\chartplayer-yyyyMMdd-HHmms.log)	记录排查信息; 日志随 exe
CoreUtil\ParityCli.cs	parity 对拍 (引擎壳/桥接覆盖/ClickByText)	自动化回归工具
CoreUtil\ChartParser*.cs	格式解析 (.osu/.mc/.sm/.aff/.adofai/.txt/.mil/.json)	支持新谱面格式
Forms\MainForm.cs	纯宿主 (引擎壳生命周期/Act 白名单/退出)	很少需要改
Forms\SettingsPanel.cs	9 分区完整设置面板	加设置项 (引擎设置行 EngineMainShell 同步)

Forms\CalibrationForm.cs	打拍校准对话框（引擎校准页/设置页共用）	校准逻辑
Forms\SkinSettings.cs / SkinDto	皮肤数据模型（颜色/字体/HudPos/ModeLayout）	皮肤/布局字段
Forms\MpEmbedPanel / MpLobbyForm	联机大厅与排行	联机功能
Play\GamePanel.cs	游玩核心：PaintCore/各模式 Draw*/ComputeLayout/布局编辑/判定接入	渲染、布局、游玩逻辑主战场
Play\EngineUi\EngineMainShell.cs	引擎壳（主菜单/设置行/选歌页/SongGrid/页游标）	壳 UI 与入口动作
Play\EngineUi\SecondaryPages.cs	引擎次级页（曲库/段位/回放/校准/玩家/我的数据/关于）	引擎化页面
Play\EngineUi\UiGlyphRuns.cs + Gdi/D2D 适配	文本测量与 emoji 口径（测量=推进=绘制三端一致）	图标-文字间距修正
Charting\ChartEditorPanel.cs	谱面编辑器（三栏：左曲目/中画布/右 7 Tab）	编辑功能
Mp\MpLeaderboard.cs	联机排行榜（已迁 BaseDirectory\PlayerData\mp_leaderboard.json）	排行存储
CoreUtil\ScreenCapture.cs	屏幕采集（F12 → 屏幕采集\）	截图功能

1.3 引擎层关键文件表（引擎源码\engine\）

文件	职责	改它做什么
MilestoneEngine.csproj	引擎类库工程	引擎引用
RhythmCore.cs	音游核心：JudgeTier/JudgementProfile（16 预设）/MaxWindowMs/窗口乘数	判定档位（见 4.2）
JudgementTracker.cs	判定追踪器（二分下界热路径/SetProfile 档位热换/事件）	判定逻辑
Ruleset.cs / RulesetFactory.cs	四大玩法规则族（Lane/Line/Ring/Path）+ 示例谱种子	玩法规则（见 4.1）
Playfield.cs	场抽象：PlayfieldLine/LineField/RingField	场几何
InputCore.cs	输入抽象：TouchInput/KeyInput/HitZone	输入
AudioClock.cs	音频时钟（Bind 引擎音频后端/采样回调驱动）	时基
EngineTime.cs / Scene.cs / SceneManager.cs / GameLoop.cs / GameObject.cs	时间/场景运行时	场景层
Tween.cs / Particles.cs / Engine3D.cs / Cam3D.cs / Transform.cs	动画/粒子/数学/相机	表现层
EngineJobs.cs	多线程并行门（ParallelMap/Reduce/Paused 降级）	并行计算
EnginePresets.cs	模式预设注册表（t57）10 项：mania4/6/8 phigros arcaea cytus osustd iidx maimai adofai	加模式预设（见 2.3/4.1）
Storyboard.cs / JudgementLog.cs / PracticeSession.cs	剧情/判定可视化/练习模式	演出与练习
ChartModel.cs / ChartValidator.cs	谱面模型（BpmTimeline/ChartData）/ 规则校验	模型与查错
NoteStyle.cs	NoteStyleBook（模式配色/描边/发光）	音符样式
StressEngine.cs / StressCore.cs / HardwareProbe.cs / ScoreBoard.cs / ParityCore.cs	压测/硬件/计分/对拍核心	工具链
App\EngineApp.cs / App\ViewportPolicy.cs	引擎宿主与多分辨率策略（FitMode/Letterbox/4096 钳制）	分辨率（见 3.5）
Ui\UiComponents.cs / Ui\UiTheme.cs / Ui\UiCanvas.cs	UI 组件（Label/Button/Grid/Stack 裁剪省略换行）/ 主题令牌 / 画布（RenderCaching 脏渲染）	UI 布局与主题
Platform\SoftwareRenderer.cs / Win32Window.cs	软渲染/窗口（SyncClient 帧缓冲/letterbox Present）	平台层
Tests\EngineChecks\	引擎自检 CLI（--stress-engine/自检断言）	回归门禁
EnginePlay\	引擎无头播放 CLI（--preset <id> --headless 预设闭环）	引擎验证
PublishPlayer\	引擎单文件 exe（MilestoneEngine.exe 发布）	引擎发布
Samples\	演示/示例（DemoRunner 等，编入游戏）	演示内容
README.md	引擎自述（每文件模块说明，权威）	引擎入门

2. 模式更改：加/改一个音游模式

2.1 完整链路（新增玩法时必须走通）

1. **注册**：源码\CoreUtil\ModeSystem.cs 的 ModelInfo 表加一行（参考既有项目）：Mode=枚举（GameMode）、Id=唯一短名（如 mania）、Display=显示名、Keys=默认键数、Tracked/Adjustable、KeyHint、格式说明。
2. **格式解析**：源码\CoreUtil\ChartParser* 增加对应扩展名/格式分支（.osu/.aff/.adofai/.txt/.mil/.json 已有各自解析器；新格式=新解析器或扩展 ChartParserExtra）。
3. **渲染**：源码\Play\GamePanel.cs 增加 Draw 函数（参考 DrawMania/DrawPhigros/DrawArcaea/DrawCytus/DrawOsuStandard/DrawIidx/DrawMaimai/DrawAdofaiReal），并在 PaintCore 的分派处接入——同一 PlayLayout 输入（Area/TopY/HitY/Ppm/LaneW...）保证多分辨率自动成立。
4. **判定**：引擎侧 Ruleset 派生（Lane/Line/Ring/Path 四族选一，或自定义），注册到 RulesetFactory；JudgementProfile 档位表加预设（RhythmCore 16 档位之一或新增）。
5. **场/输入**：Playfield 场族（LineField/RingField/...）+ InputCore 命中区（HitZone 矩形/圆/环）。
6. **编辑器**：ChartEditorPanel 的模式分派（有轨/无轨范式；无轨按 RPE 白线+事件；有轨按列式网格）。
7. **示例与校验**：ChartValidator 模式规则 + 示例谱（输出产物\示例内容\Milestone示例*.mil / 测试格式\）+ EngineChecks 断言。
8. **验证**：见 5.4 标准验收流程（selfcheck/parity/shellshot/还原度对比）。

2.2 如何改已有模式（小改）

- **改键数/默认值**：ModelInfo (Keys/Adjustable) + GamePanel 键数逻辑 + ChartParser 端校验。
- **改判定窗口**：JudgementProfile 预设（引擎\engine\RhythmCore.cs）或运行期 JudgementTracker.SetProfile 热换 + WindowMultiplier（0.25~4 乘算，×1.0 零差异——设置页「判定基准」）。
- **改流速/缩放/方向**：谱面事件（zoom/noteSpeed/scroll, ComputeLayout 求值）+ 布局元素 field.speed（见 文档精选\layout-custom-design.md）。
- **改数值风格**：NoteStyleBook（引擎\engine\NoteStyle.cs）——每模式配色/描边/发光；皮肤 NoteColors 亦可覆盖。

2.3 模式预设（EnginePresets, t57）

- 位置：引擎源码\engine\EnginePresets.cs（静态注册表，10 项：mania4/6/8 phigros arcaea cytus osustd iidx maimai adofai）。
- 每项：Id/Display/Field (Lane/Line/Ring/Path) /Keys + 生成示例谱种子（RulesetFactory.BuildSampleChart, 音符数/长条/滑星特征可调）。
- 引擎侧闭环验证：引擎源码\engine\EnginePlay --preset <id> --headless（无头跑预设 → ACC/轨迹输出）；ToDocTable() 生成预设表文本。
- 游戏内入口：引擎菜单「AI 演示」等按预设驱动（AiEngine 规则式，不接 LLM）。

3. UI 更改：改文字 / 布局 / 主题 / 图标

3.0 三条铁律（先读）

1. **文字只改 UiText.cs**（源码\CoreUtil\UiText.cs）——每键带注释「原文出处」，严禁在场景/面板代码里写死用户可见字符串。
2. **布局只改坐标与自适应公式**：所有页面布局由 UiMeasure/UiTheme.Scaled 自适应公式决定，不要写绝对像素当修复——多分辨率（1280×800→4K+DPI）会崩。
3. **每改一处一构建一截图核对**：用 输出产物\bin\Release\net8.0-windows\win-x64\Milestone.exe --shellshot <目录>（引擎壳三页离屏渲染）或 --menushot 快速取证，别纯目测。

3.1 改文字

- 例：主菜单「▶ 开始游戏」在 UiText.cs:25 (MenuPlayGame)；功能按钮 18 项紧随其后；提示行 MenuHint；设置分区/次级页/编辑器文案各区段都有注释索引。
- 改后：引擎壳/引擎演示/parity 基线同源 (UiText 常量)，改一处全界面生效；**注意 parity-text-baseline.json 基线：若改了作为基线键的文案，parity -1 会报缺——按 5.4 同步基线或选择非基线键。**

3.2 改布局 (页面元素)

- 壳页面布局：源码\Play\EngineUi\EngineMainShell.cs / SecondaryPages.cs (场景代码：UiPanel/UiButton/UiLabel/UiStackLayout/UiGridLayout 的 X/Y/Width/Height/FontSize/Align)。
- 组件基类：引擎源码\engine\Ui\UiComponents.cs (Label Clip/Ellipsis/WordWrap/AutoShrinkFont、Button/Card、Stack/Grid ClipChildren+ChildAlign、Canvas ToVirtual/RenderCaching)。
- 自适应：UiMeasure (按钮宽/字号 FitFont/MaximumSize) + UiTheme.Scaled(k) (缩放令牌) + ViewportPolicy (引擎壳多分辨率，含 letterbox 逆变换)；编辑器画布最小 640 自动折叠等 (editor-trilab.md)。
- 改坐标采用「参考值+公式」：如文件夹页 48→48+Δ 的既有坐标表 (Visual 系列文档)，改完跑 --shellshot 三个尺寸 (1280/1920/物理 DPI) 比对。

3.3 改主题

- 引擎源码\engine\Ui\UiTheme.cs: Bg (根背景) /BgDeep/Surface/Surface2/Glass/GlassDark/TextPrimary/Secondary/Muted/Accent/Accent2/Success/Warning/Danger/Info/BtnBg.../CardBg...；间距 S1-S4、圆角 R1-R3 (圆角/字号段)；Scale 令牌与 Scaled(k)。
- 改法：调整任一字段初值 (RgbColor 十六进制/ARGB)；如需整套换肤，继承 UiTheme.Default 或 new 后覆盖字段 (引擎壳主题切换 CycleTheme 走同一路径)。
- 注意：主题=颜色/字号/间距；**布局与几何 (位置/倍率) 归 布局编辑器 (Skin.ModeLayout, 见 layout-custom-design.md)，皮肤文件=颜色细节 (SkinSettings) ——三层别混。**

3.4 改图标与 emoji 口径

- 文本 emoji 推进=UiGlyphRuns 口径 (源码\Play\EngineUi\UiGlyphRuns.cs: CodePointCount/MeasureRuns, 代理对按完整码点、估宽=码点×字号×1.15 或真实 advance, 测量=推进=绘制三端同口径；GdiDrawAdapter/D2DDrawAdapter 同规则)。
- 修改任何图标文本前：确认 测量 与 推进 同口径 (否则图标压字/间距错误——历史 P0 教训)。
- 画制类图标 (皮肤资源如 osu_default 的 approachcircle/hitcircle png) 在 输出产物\示例内容\皮肤\osu_default\ (@2x 提供双倍清晰度)，皮肤加载按 SkinSettings。

3.5 分辨率/刷新率 (引擎层)

- ViewportPolicy (引擎源码\engine\App\ViewportPolicy.cs)：FitMode/Letterbox/DpiContext/ResolutionProfile/StartupSize——渲染分辨率=物理客户区×renderScale (4096 钳制)；壳层 1280×800 位图时代已废除。
- FpsGovernor (源码\Play\FpsGovernor.cs)：刷新率表 (-1 跟随屏幕/50..500/0 无限制 + 自定义 1..2000Hz, 0.5ms 最小帧时)；TickFps 时间化迟滞 1.5s/4s；与 AutoAdapt 单一仲裁。
- 改这些=引擎层配置，别在 UI 层二次实现。

4. 引擎改动：Ruleset / 判定 / 渲染 / 引擎工具 CLI

4.1 Ruleset（玩法规则族）

- 基类：Ruleset.cs (abstract; ChartContext + 场/输入/判定无头组合)；派生四族：Lane（轨）、Line（线）、Ring（环）、Path（路径）——RulesetFactory 注册。
- 新增玩法：派生 Ruleset → 注册 → EnginePresets 加预设 → EnginePlay --preset 无头闭环验证（无头可跑=规则自治）。
- 样例种子：RulesetFactory.BuildSampleChart(mode, seed)——音符数/长条/滑星按预设元数据生成，用于演示/测试/编辑器空谱。

4.2 判定

- RhythmCore.cs: JudgeTier（档位）/JudgementProfile（任意档位+窗口，16 预设：Arcaea/Phigros/IIDX/Cytus/ADOFAL/mania(OD)...）/MaxWindowMs（乘 WindowMultiplier 0.25~4）/Evaluate。
- JudgementTracker.cs: 谱面→判定闭环（自动漏判/按下命中/长条保持；Press* 已二分下界化→热路径 0 分配）；SetProfile 运行期档位热换（判定基准下拉=该 API）；Profile/JudgedDetail 事件；WindowStartMs 段界跳过（练习循环）。
- 练习/可视化：PracticeSession.cs（段循环/幽灵回放/Rate 0.5-1.5）+ JudgementLog.cs（滚动 200 判定均值/标准差/9 桶直方图/轨热图）。
- 改判定**必须**同时加 EngineChecks 断言（引擎源码\engine\Tests\EngineChecks\——确定性/零分配/热换/窗口乘数 ×1.0 防回归硬断言）。

4.3 渲染（IUiDraw 适配器约定）

- 宿主经 IUiDraw 绘制（DrawLine/DrawRect/FillRect/Text/MeasureText/PushClip...）；**Text 与 MeasureText 必须同口径**（emoji 估宽统一；D2D 用 TextLayout 真实 advance，GDI 用估宽——两侧已对齐）。
- UiCanvas: DesignW/H + SetViewport（虚拟坐标→物理）+ RenderCaching（脏渲染）+ ToVirtual（letterbox 逆变换）；离屏取证（--shellshot）前 Invalidate 强制出图。
- 性能护栏（历史基线）：每帧零分配热路径（引擎壳 82.6B/帧）、字体/刷/笔静态缓存、UiElement 测量缓存有界（Render 虚拟化）、脏渲染默认开。

4.4 引擎工具 CLI（引擎源码\engine 各工程）

工程	用途	示例
Tests\EngineChecks	引擎自检/压测 CLI	dotnet run --project ..\引擎源码\engine\Tests\EngineChecks -c Release -- --stress-engine <outDir>（res.json 断言全 PASS）
EnginePlay	无头播放/预设闭环	dotnet run --project ..\引擎源码\engine\EnginePlay -c Release -- --preset mania4 --headless（ACC/轨迹输出；--list 预设表）
PublishPlayer	引擎单文件发布	dotnet publish -c Release -r win-x64 -o <out>（MilestoneEngine.exe）
HelloEngine	最小引擎示范	入门参考

5. 打包与命令

5.1 构建/发布命令表（新树工作目录）

用途	命令（PowerShell，工作目录=游戏源码\）
游戏构建	dotnet build -c Release （产物 输出产物\bin\Release\net8.0-windows\win-x64\Milestone.exe）
游戏发布单文件	dotnet publish -c Release -r win-x64 -o ..\输出产物\发布\ （PublishSingleFile+SelfContained+IncludeNativeLibrariesForSelfExtract；发布件=Milestone.exe 单文件+skin.json；首次启动自动建 Chart/Player/PlayerData/Replay/Log/屏幕采集/人类试玩）
引擎发布	cd ..\引擎源码\engine\PublishPlayer; dotnet publish -c Release -r win-x64 -o <输出目录>
引擎自检	cd ..\引擎源码\engine\Tests\EngineChecks; dotnet run -c Release （全绿 exit 0）
预设闭环	cd ..\引擎源码\engine\EnginePlay; dotnet run -c Release -- --preset <id> --headless
自定义打包	输出产物\单文件\重打包（可用 --pack CLI，见 5.2）

说明：csproj 已迁移 OutputPath=..\输出产物\bin\ 与 ProjectReference=..\引擎源码\engine\MilestoneEngine.csproj；引擎 Samples 经 Include 编入游戏；**不要改回旧路径。**

5.2 dev CLI 速查（Program.cs）

参数	作用
--selfcheck	全量自检（构建期门禁，exit 0=过）
--parity -1	parity 对拍（壳三页渲染/桥接覆盖/ClickByText；-o 指定输出）
--shellshot <dir>	引擎壳三页离屏截图（menu/songs/settings）
--menushot <dir>	主菜单截图（壳渲染）
--edshot / --edsim	编辑器截图/模拟
--modetest	全模式回归
--noteshot / --shotdemo / --storyshot / --multishot / --foldershot / --uisceneshot	各定向取证
--perfctest / --stresstest / --stress-engine / --gamestress	性能/压力（res.json）
--engineerun / --fpsprobe / --wincount	引擎运行探测
--pack [--out]	单文件打包（发布件/外带素材组装；--out 指定输出）
--humantest	人类模拟游玩（试玩用）
--composertest	回环作曲自检
--aidetect	AI 检测（本地 AI 已禁用=输出禁用说明，规则检查仍可用）
--paritycell	单元格对拍

5.3 桌面快捷方式（发布件）

```
~~~
$ws = New-Object -ComObject WScript.Shell

$Ink = $ws.CreateShortcut("$env:USERPROFILE\Desktop\Milestone.Ink")

$Ink.TargetPath = "D:\Users\etgya\Desktop\milestone\输出产物\发布\Milestone.exe"

$Ink.WorkingDirectory = "D:\Users\etgya\Desktop\milestone\输出产物\发布"

$Ink.Save()

~~~
```

- 快捷方式指向**发布件**（单文件），运行数据全部在 exe 旁。

- 若使用 输出产物\单文件\MilestonePlayer.exe（额外打包），同法重建。

5.4 标准验收流程（改代码后按序跑）

- 1. dotnet build -c Release → 0 警告 0 错误
- 2. Milestone.exe --selfcheck → exit 0
- 3. EngineChecks 全绿（引擎改动必跑；含 StressEngine 断言可选 --stress-engine 到 dev\）
- 4. Milestone.exe --parity -1 → 全绿（UI 文案改动注意基线同步）
- 5. --shellshot <目录> → 三页 PNG 目视/截图核对（文字重叠/截断回归）
- 6. 人工启动走查（选歌→游玩→设置→退出）→ 进程干净退出

6. 文档索引与路径映射

6.1 精选索引（Top 权威篇，编辑指南\文档精选\）

文档	用途
folder-structure-design.md	四目录设计（本布局依据）
t53-设计定稿-修正.md + legacy-removal-design.md	非引擎化删除设计（MainForm 纯宿主）
editor-trilab.md	编辑器三栏规范（t5 依据）
layout-custom-design.md	布局编辑器全玩法自定义设计（t63 依据）
engine-unity-arch.md	引擎架构说明
engine-tool-design.md / engine-presets-data.md / engine-capability-list.md	引擎工具化设计与能力清单
还原度闭环*_最终总结.md	各模式还原度基准（改渲染前必读）
主菜单现代UI评审（t60 系列）	UI 视觉评审
视觉终验* / visual-*	视觉验收基线
stress-matrix.md / stress-*-report.md	压测矩阵与报告
功能复刻对比清单.md	功能对照

6.2 路径映射（旧→新，历史文档正文仍写旧路径，按此表换算）

旧	新
源码\	游戏源码\源码\
引擎\engine\	引擎源码\engine\
其他\docs\协作	编辑指南\文档精选\
构建产物\obj\	输出产物\dev\
发布\	输出产物\发布\
其他\Chart\	输出产物\示例内容\Chart\（发布件内 Chart\ 为运行时副本）
bin\	输出产物\bin\
obj\（根）	游戏源码\obj\
其他\段位文件\	输出产物\示例内容\段位文件\

7. 附录：运行/调试速查与常见坑

- 启动：双击 输出产物\发布\Milestone.exe；或 输出产物\bin\Release\net8.0-windows\win-x64\Milestone.exe。
- 日志：exe 旁 Log\chartplayer-yyyyMMdd-HHmms.log（Logger.LogDir=BaseDirectory\Log）；菜单「📄 打开日志」=打开该目录。
- 配置：%LOCALAPPDATA%\ChartPlayer\config.json（ChartsFolder 等；t62 已加一次性迁移标记 ChartsFolderMigrated）。
- 皮肤：Player\skin.json（程序目录）优先，其次 exe 目录 skin.json。
- 常见坑：
 1. 改了 UiText 但界面没变 → 编译产物没更新/用了旧 bin（确认 输出产物\bin 时间戳）。
 2. 文字截断/重叠 → 没走 UiMeasure/AutoShrinkFont/Ellipsis 三件套（参考 SectionHeader/PathLabel 实现）。
 3. emoji 压字 → UiGlyphRuns 测量≠推进（检查适配器是否同口径）。
 4. 新模式谱面不显示 → ModelInfo 键数/解析器/GamePanel 分派三处至少一处漏。
 5. parity 红 → 先看 基线 diff（parity-text-baseline.json），确定是有意改动再同步，禁止静默改基线。
 6. 打包后启动黑屏/找不到资源 → 内容根约定：发布件 = exe 旁 Chart\；皮肤 png 用 @2x 时注意命名。

8. 素材与内容清单（实测目录结构）

8.1 输出产物\示例内容\（素材库，发布件 Chart\ 副本源）

子目录	内容	用途
Milestone示例\	Mania 4K / Phigros / Arcaea / Cytus / IIDX / 多模式 示例.mil	九模式最小示例谱（零基础任务 5 的模板）
exports\	Mania 4K.mil / maimai.mil（导出样本）	导出格式参照
实例谱面\	phigros\（一番の宝物/Undertale/IF_Infinity 三套带音频）	完整曲目实例
测试格式\	每格式最小样例（mania_test.osu / phigros_*.json / arcaea_*.aff / cytus_*.txt / adofai_*.adofai / taiko_test.osu / iidx_test.mil / mil 系多份）	格式参考与回归素材
段位文件\	Dan 系列 .mc/.osu（00th~exf + REFORM/Malody/wds0 等 100+ 文件）	段位挑战内容（发布件副本进 Chart\）
段位挑战\	osu!mania 4K Dan	AI 段位等级素材
皮肤 \osu_default\	hitcircle/approachcircle/slider*/spinner*（含 @2x）+ skin.ini + LICENCE	osu 皮肤资源
其他	Player\ai_train.json（AI 训练参数样例）、library_rating.json（评级数据样例）	数据样例

8.2 谱面格式速查

格式	模式	说明
.mil	全部	Milestone 原生 JSON（format: milestone-1；mode/title/artist/bpm/offset/audio/keys/notes[{t,c,e,type}]; 多模式=parts 数组）
.osu	Mania/Taiko/Catch/osu!std	osu 格式（支持 .osu 多种模式；Dan 段位常用）
.mc / .sm / .ssc / .qua	Mania	Malody/SM/Quaver 兼容
.aff	Arcaea	Arcaea 原生
.adofai	ADOFai	路线格式
.txt	Cytus / maimai	自定义文本格式

.json	Phigros	RPE/Phigros 事件格式
-------	---------	------------------

8.3 运行时数据（发布件 exe 旁，BaseDirectory）

目录/文件	说明
Chart\	谱面（默认曲库；发布件内置示例副本）
PlayerData\	playerdata.json（性能/存档） + mp_leaderboard.json（联机排行，t62 已迁入）
Player\	player.json / skin.json / ai_train.json
Replay\	回放
Log\	会话日志（chartplayer-yyyyMMdd-HHmss.log）
屏幕采集\	F12 截图
人类试玩\	人类模拟存档
skin.json（exe 旁）	皮肤后备（Player\ 优先）

8.4 修改渲染/判定的函数索引（按函数名搜索，勿依赖行号）

绘制：PaintCore / DrawMania / DrawMania3D / DrawPhigros / DrawArcaea / DrawCytus / DrawOsuStandard / Drawlidx / DrawMaimai / DrawAdofai / DrawAdofaiReal / DrawLoopComposer / DrawHud。

布局：ComputeLayout / ComputeLayoutFor / PlayAreaRect / HudBox / HitLineBox（布局编辑器交互）。

判定：JudgementTracker.Press/Release/Query / RhythmCore.Evaluate / JudgementProfile。

引擎场景：SceneManager.Tick / SceneTransition / GameLoop.Tick / RulesetFactory.Create。